

SystemTap

If programming is an art, then debugging is even more so. To be a good programmer, one must master debugging. We have seen some good methods of kernel debugging, e.g., gdb, kgdb, kprobes, etc, but none is as dynamic a tool as SystemTap, a probing and tracing tool that lets you analyse a Linux kernel's activity deeply, at run-time.



SystemTap can probe system calls and kernel functions at run-time, and can examine variables in functions. There is no need to change the kernel source code to insert instrumentation, and then re-compile and install it. Simple scripts can be developed to probe the kernel at run-time.

Installation

SystemTap can be installed directly via the Synaptic package manager for Ubuntu/Debian, or Yum for CentOS, Fedora and Red Hat. However, I always like to download the source code and compile it, with the following steps.

Download the source code from <http://sourceware.org/systemtap/releases/>. I fetched the latest, *systemtap-1.6.tar.gz*. Extract the tar ball with `tar -zxvf systemtap-1.6.tar.gz`, and change to the extracted directory (`cd systemtap-1.6`). Run `./configure`. If this fails at any point (an error for a missing package) then you have to install that package (usually with `yum/apt-get`). Once `configure` is done, compile it with `make`. It will take only a couple of minutes; after it's done, look for the output binary, *stap*. Now run `make install` to install SystemTap. You could also add your SystemTap directory to your `PATH` variable, to directly run *stap* without installing.

Let's test SystemTap

This is a sample SystemTap script, which I will describe later on. First run it (as the root user, else you will get the error "Warning: /usr/local/bin/staprun is not executable (Permission denied) ...").

```
# cat hello.stp
probe begin
{
    printf ("hello world\n");
    exit {};
}

# stap hello.stp
hello world
```

Now, I will cover some script syntax and background details. Fundamentally, SystemTap is based on *event and handler*

concepts. In Qt, .NET and other frameworks, this method is used for both interactive and non-interacting actions. When an event occurs, its corresponding handler will be executed. In SystemTap, when a specified event occurs, the Linux kernel runs the handler, and then resumes normal execution. Events are of two types, synchronous and asynchronous. Typical synchronous events are system call execution, entering and exiting a function, functions in a kernel file, etc. Asynchronous events are timers, jiffies, etc.

Handlers are written in SystemTap's script language. The format of a SystemTap script is as follows:

```
probe event { statements }
```

In a single SystemTap script, multiple events can be used. The statements to be executed for each event are enclosed in `{ }` braces.

When a SystemTap script is executed, the script is translated into C; the C compiler compiles the code to create a kernel loadable module (.ko). During the module insertion, all the events are initialised in the kernel. So when an event occurs, the corresponding handler will be run. When `exit()` is executed, the module is unloaded. To confirm this, you can omit `exit()` from the sample script and run `stap hello.stp`. Next run `lsmod` in another terminal, to list the loaded modules. At the top of the list, you will find the *stap* module entry. Now, to exit the script, press `Ctrl+C`, which will unload the module.

Let us now try different events that we can use in SystemTap.

Probing a system call

```
# cat exec.stp
probe syscall.execve
{
    printf ("%s(%d) execve (%s)\n", execname(), pid(), argstr)
}
probe syscall.exit
{
    printf ("%s(%d) exit (%s)\n", execname(), pid(), argstr)
}
```

The syntax for system-call probing is `'probe syscall.syscall-`